

Configuration Management using Spring Cloud Config Server

(Native File System Configuration)

Learning Objectives

After completing this module, you will be able to:

- Understand why configuration management becomes difficult in a Microservices architecture.
 - Explain the concept of Externalized Configuration.
 - Understand the role of Spring Cloud.
 - Create a Spring Cloud Config Server.
 - Configure a Native (Local File System) Configuration Repository.
 - Configure microservices to retrieve configuration from Config Server.
 - Understand how configuration is loaded during application startup.
 - Explain the benefits of centralized configuration management.
-

1.1 Why Configuration Management?

When building a monolithic application, all configuration is usually stored within a single application.

Typical configuration includes:

- Database URL
- Username
- Password
- Server Port
- Logging Level
- API Keys
- Mail Configuration

Example:

application.properties

spring.datasource.url=...

spring.datasource.username=root

spring.datasource.password=****

server.port=8080

Managing configuration is simple because there is only one application.

Configuration in Microservices

An e-commerce application usually consists of several independent services.

Example:

Product Service

Customer Service

Order Service

Inventory Service

Payment Service

Notification Service

Each service has its own configuration file.

product-service

application.properties

customer-service

application.properties

order-service

application.properties

Each service has different:

- Database
- Port Number
- Logging Level

- Kafka Configuration
- API Keys
- Mail Settings

As the number of services grows, maintaining configuration separately becomes increasingly difficult.

Real-World Scenario

Suppose your organization has:

- 35 Microservices
- 4 Environments
- Development
- Testing
- UAT
- Production

That means:

35 Services × 4 Environments

= 140 Configuration Files

Imagine changing the database password.

Without centralized configuration, you would have to update all 140 configuration files individually.

This is time-consuming, error-prone, and difficult to maintain.

Problems Without Centralized Configuration

Suppose the database password changes.

Without Config Server:

Update Product Service

↓

Update Customer Service

↓

Update Order Service

↓

Update Inventory Service



Update Payment Service



Update Notification Service

Every service must be restarted after updating its own configuration.

Now imagine changing:

- Database URL
- Kafka Server
- Mail Server
- API Keys
- Logging Level

The maintenance effort quickly becomes overwhelming.

Why Do We Need Centralized Configuration?

A better approach is to keep configuration in one central location.

Instead of every service maintaining its own configuration, all services retrieve configuration from a central Config Server.

Benefits include:

- One place to manage configuration
- Easy maintenance
- Consistent configuration
- Reduced duplication
- Easier deployments
- Better security

1.2 What is Externalized Configuration?

Externalized Configuration means keeping configuration outside the application.

Instead of packaging configuration inside the application, configuration is stored separately.

Traditional Approach

Application

↓

application.properties

Externalized Configuration

Application

↓

Config Server

↓

Configuration Repository

Whenever the application starts, it requests its configuration from the Config Server.

Why Externalize Configuration?

Suppose Product Service uses:

server.port=8081

database=productdb

Instead of storing this inside the application,
the configuration is stored in a central repository.

Whenever Product Service starts,

it simply requests:

"Please provide my configuration."

The Config Server sends the required configuration.

No configuration is packaged inside the application.

Advantages of Externalized Configuration

✓ One source of configuration

- ✓ Easier maintenance
 - ✓ Better security
 - ✓ Environment-specific configuration
 - ✓ Easy updates
 - ✓ No need to rebuild applications after configuration changes
-

1.3 Introduction to Spring Cloud

Spring Boot helps us build Microservices.

Spring Cloud helps us manage Microservices.

Spring Cloud provides several components including:

- Config Server
- Eureka Service Discovery
- API Gateway
- Load Balancer
- Circuit Breaker
- Distributed Configuration

In this course we will use:

- Config Server
 - Eureka
 - Gateway
 - Resilience4j
-

1.4 Spring Cloud Config Server

Spring Cloud Config Server is a centralized configuration server.

Instead of every application storing configuration independently, all configuration is maintained in one place.

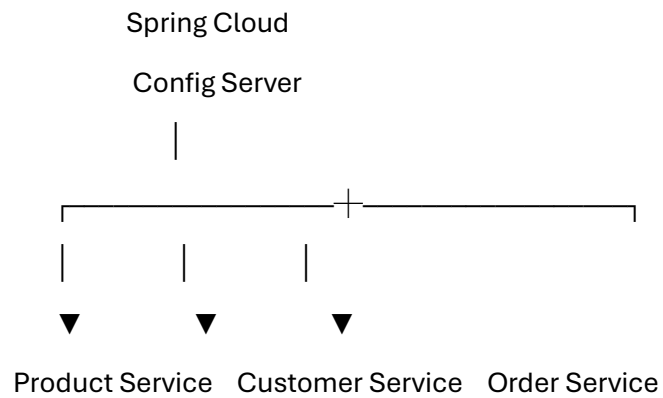
Architecture

Local Configuration Folder

(C:\config-repo)

|

|



The Config Server acts as a central configuration provider for all microservices.

How Config Server Works

Step 1

Administrator updates configuration files stored in:

C:\config-repo

↓

Step 2

Config Server reads configuration from the local folder.

↓

Step 3

Microservice starts.

↓

Step 4

Microservice requests its configuration.

↓

Step 5

Config Server sends the configuration.

↓

Step 6

Application starts successfully.

Why Use a Local Configuration Repository?

For classroom training, storing configuration on the local file system provides several advantages:

- Easy to understand
- No Git knowledge required
- Quick setup
- Easy debugging
- Ideal for development and demonstrations

Example directory:

C:\config-repo

application.properties

product-service.properties

customer-service.properties

order-service.properties

Each microservice has its own configuration file, while common settings can be shared through a common configuration file.

Config Server Configuration

Configure Config Server to use the Native File System.

application.properties

server.port=8888

spring.application.name=config-server

spring.profiles.active=native

spring.cloud.config.server.native.search-locations=file:///C:/config-repo

This tells Config Server to retrieve configuration from the local directory instead of a Git repository.

Configuration Loading Process

Customer Service Starts



Requests Configuration



Config Server



Reads Configuration

from C:\config-repo



Returns Configuration



Customer Service Starts Successfully

The microservice does not need to know where the configuration is stored. It only communicates with the Config Server.

Advantages of Config Server

- Centralized configuration management
- Simplified maintenance
- Consistent configuration across services
- Environment-specific configuration
- Reduced duplication
- Better security
- Easier DevOps integration
- Improved scalability

Best Practices

- Keep configuration outside the application.
- Store configuration in a centralized location.
- Maintain separate configuration for Development, Testing, and Production.
- Do not hardcode passwords.
- Use common configuration files for shared settings.
- Create service-specific configuration files for individual services.
- Use meaningful application names.

Service Discovery with Eureka Server

Learning Objectives

After completing this module, you will be able to:

- Understand why Service Discovery is required in a Microservices architecture.
- Explain the drawbacks of hardcoded service URLs.
- Differentiate between Static and Dynamic Service Discovery.
- Understand the architecture of Eureka Server.
- Explain Service Registration and Service Registry.
- Register Product Service and Customer Service with Eureka Server.
- Verify service registration using the Eureka Dashboard.
- Understand how microservices communicate using service names.

2.1 Introduction to Service Discovery

In a **Monolithic Application**, all modules are packaged and deployed as a single application. Communication happens through direct Java method calls.

User

|



Monolithic Application

|— Product Module

└— Customer Module

└— Order Module

└— Inventory Module

Since all modules are inside one application, locating another module is never a problem.

Communication in Microservices

In a Microservices architecture, every business capability is developed as an independent application.

Example:

Product Service

Customer Service

Order Service

Inventory Service

Payment Service

Notification Service

Each service:

- Runs independently
- Has its own database
- Has its own port
- Can be deployed independently

Because every service runs separately, they must communicate over the network.

Example

Suppose a customer places an order.

The Order Service needs:

- Customer Information

- Product Information
- Inventory Status

Since this data belongs to different services, Order Service must communicate with them.

Order Service

|

└───▶ Customer Service

└───▶ Product Service

└───▶ Inventory Service

This communication happens using REST APIs.

The Problem

Suppose Product Service is running on:

`http://localhost:8081`

Customer Service calls

`http://localhost:8081/api/products`

Everything works correctly.

Now suppose Product Service is restarted.

Instead of

8081

it starts on

9091

Customer Service is still trying to call

`localhost:8081`

The request fails.

Why Hardcoded URLs Are a Problem

Imagine an application containing:

- Product Service
- Customer Service
- Order Service
- Inventory Service
- Payment Service
- Notification Service

Every service stores the address of every other service.

If Product Service changes its port,
all dependent services must also be modified.

As the number of services increases, maintaining these URLs becomes difficult.

Problems with Hardcoded URLs

Hardcoded URLs introduce several challenges:

- Manual configuration updates
- Difficult maintenance
- Tight coupling between services
- Poor scalability
- Frequent deployment changes
- Increased chances of configuration errors

Real-World Scenario

Suppose your application has:

- 20 Microservices
- 5 Instances of each service

The Product Service may be running as:

localhost:8081

localhost:8082

localhost:8083

localhost:8084

localhost:8085

Which one should Customer Service call?

Managing these addresses manually is impractical.

2.2 What is Service Discovery?

Service Discovery is a mechanism that allows microservices to locate one another dynamically.

Instead of remembering an IP address or port number,

services communicate using a logical **Service Name**.

Instead of

`http://localhost:8081/api/products`

we use

`http://PRODUCT-SERVICE/api/products`

The actual network address is determined automatically by Eureka Server.

Advantages of Service Discovery

- No hardcoded URLs
- Automatic service lookup
- Easier deployment
- Better scalability
- Independent services
- Fault tolerance
- Simplified maintenance

2.3 Static vs Dynamic Service Discovery

Static Discovery

Customer Service

|



localhost:8081

Characteristics

- Manual configuration
- Fixed URL
- Difficult to scale
- Suitable only for small applications

Dynamic Discovery

Customer Service

|



Eureka Server

|



Product Service

Characteristics

- Automatic discovery
- Dynamic resolution
- Easy scaling
- Cloud friendly
- Production ready

Static vs Dynamic Comparison

Feature	Static Discovery	Dynamic Discovery
URL	Hardcoded	Service Name
Configuration	Manual	Automatic
Scalability	Difficult	Easy
Maintenance	High	Low
Deployment	Complex	Simple
Cloud Support	Limited	Excellent

2.4 What is Eureka?

Eureka is a **Service Discovery Server** developed by Netflix and integrated into Spring Cloud.

It acts as a **telephone directory** for Microservices.

Whenever a service starts,

it informs Eureka:

"I am available."

Whenever another service needs it,

it asks Eureka:

"Where is Product Service?"

Eureka returns the current address.

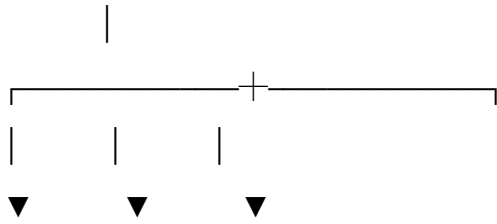
Responsibilities of Eureka

Eureka is responsible for:

- Registering Microservices
 - Maintaining Service Registry
 - Discovering available services
 - Monitoring service health
 - Removing unavailable instances
 - Returning current service locations
-

Eureka Architecture

Eureka Server



PRODUCT-SERVICE CUSTOMER-SERVICE ORDER-SERVICE

Every service registers itself with Eureka during startup.

2.5 Service Registration

When Product Service starts,

it automatically sends its details to Eureka.

Information includes:

- Application Name
- Host Name
- IP Address
- Port Number
- Instance ID
- Status

Example

Application : PRODUCT-SERVICE

Host : localhost

Port : 8081

Status : UP

Eureka stores this information in its registry.

2.6 Service Registry

The Service Registry is the database maintained by Eureka.

Example

Service	Port	Status
---------	------	--------

PRODUCT-SERVICE	8081	UP
-----------------	------	----

CUSTOMER-SERVICE	8082	UP
------------------	------	----

ORDER-SERVICE	8083	UP
---------------	------	----

Whenever a service starts, stops, or changes its location,
the registry is automatically updated.

2.7 Service Discovery Workflow

Product Service Starts

|

Registers with Eureka

|

Customer Service Starts

|

Requests PRODUCT-SERVICE

|

Eureka Returns Address

|

Customer Service Calls Product Service

No hardcoded host names or ports are required.

Complete Request Flow

Client

|

Customer Service

|

Eureka Server

|

Returns Product Service Address

|

Product Service

|

JSON Response

|

Customer Service

|

Client

This entire process is transparent to the client.

2.8 Eureka Dashboard

Spring Cloud Eureka provides a web dashboard to monitor registered services.

Open

`http://localhost:8761`

You should see:

Applications

PRODUCT-SERVICE

CUSTOMER-SERVICE

Status : UP

The dashboard confirms that services have successfully registered.

Why Use Application Names?

Instead of

`localhost:8081`

use

PRODUCT-SERVICE

Benefits:

- No dependency on port numbers
 - Easier deployment
 - Better scalability
 - Supports multiple service instances
 - Supports cloud environments
-

Best Practices

- Register every microservice with Eureka.
- Use meaningful application names.

- Never hardcode IP addresses.
 - Use service names for communication.
 - Monitor services using the Eureka Dashboard.
 - Keep Eureka highly available in production.
 - Enable health checks using Spring Boot Actuator.
-

Real-World Example

Imagine a shopping website during a festival sale.

The Product Service is overloaded.

The cloud platform automatically creates additional Product Service instances.

PRODUCT-SERVICE

Instance 1

Instance 2

Instance 3

Instance 4

Eureka automatically updates the registry.

Customer Service continues to use:

<http://PRODUCT-SERVICE/api/products>

without any code changes.

3.1 Customer Service Overview

In our E-Commerce application, the Customer Service is responsible for managing customer-related information.

Typical responsibilities include:

- Register a Customer
- Retrieve Customer Details
- Update Customer Information

- Delete Customer
- Search Customers
- Consume Product Service APIs

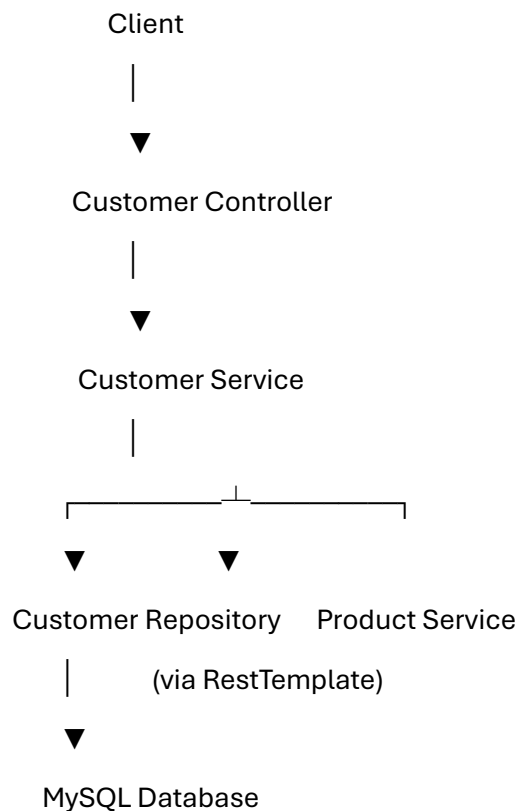
Unlike a monolithic application, the Customer Service owns its own database and never accesses another service's database directly.

Responsibilities of Customer Service

Customer Service manages:

- Customer Registration
 - Customer Profile
 - Customer Address
 - Customer Contact Details
 - Customer Validation
 - Product Lookup (through Product Service)
-

Customer Service Architecture



The controller never communicates directly with the database.

All business logic resides inside the Service layer.

Layered Architecture

Every Spring Boot Microservice in this course follows the same architecture.

Controller

|

Service

|

Repository

|

Database

Advantages:

- Easy Maintenance
- Better Testing
- Loose Coupling
- High Reusability
- Clear Separation of Responsibilities

Project Structure

Create the project using Spring Initializr.

customer-service

|

├── controller

├── service

└─ repository
└─ entity
└─ exception
└─ config
└─ model
└─ CustomerServiceApplication

Each package has a specific responsibility.

Required Dependencies

Add the following dependencies:

- Spring Web
- Spring Data JPA
- Validation
- MySQL Driver
- Eureka Discovery Client
- Config Client
- Spring Boot Actuator
- Springdoc OpenAPI

Customer Database Design

Database Name

customerdb

Customer Table

Column	Type
--------	------

id	BIGINT
----	--------

first_name	VARCHAR
------------	---------

last_name	VARCHAR
-----------	---------

email	VARCHAR
-------	---------

phone	VARCHAR
-------	---------

Column	Type
city	VARCHAR

Entity Relationship

Customer

+-----+

id

firstName

lastName

email

phone

city

+-----+

Customer Entity

The Customer entity represents one customer record.

Recommended fields

id

firstName

lastName

email

phone

city

Validation Rules

Apply Bean Validation annotations.

Recommended validations:

- First Name cannot be blank
- Last Name cannot be blank
- Email must be valid
- Phone cannot be blank
- City cannot be blank

Benefits:

- Prevents invalid requests
- Reduces database errors
- Improves API quality

Repository Layer

Create

CustomerRepository

Extend

JpaRepository<Customer, Long>

Spring Data JPA automatically provides:

- save()
- findAll()
- findById()
- deleteById()
- existsById()
- count()

No SQL needs to be written for basic CRUD operations.

Service Layer

The Service Layer contains the application's business logic.

Responsibilities include:

- Save Customer
 - Retrieve All Customers
 - Retrieve Customer By ID
 - Update Customer
 - Delete Customer
 - Validate Business Rules
 - Call Product Service
-

Why Service Layer?

Instead of

Controller

↓

Repository

Use

Controller

↓

Service

↓

Repository

Advantages:

- Cleaner Controller
- Better Reusability
- Easier Unit Testing
- Business Logic Centralization

Controller Layer

Create

CustomerController

Annotate with

@RestController

@RequestMapping("/customers")

The controller receives HTTP requests and forwards them to the Service layer.

REST Endpoints

Method	Endpoint	Description
GET	/customers	Retrieve All Customers
GET	/customers/{id}	Retrieve Customer By Id
POST	/customers	Create Customer
PUT	/customers/{id}	Update Customer
DELETE	/customers/{id}	Delete Customer

Request Flow

Client

↓

CustomerController

↓

CustomerService



CustomerRepository



MySQL



JSON Response

Every request follows the same architecture.

Configuring Customer Service

Instead of storing configuration locally,

Customer Service retrieves configuration from Config Server.

Customer Service



Config Server



C:\config-repo



Configuration Returned

This ensures centralized configuration management.

application.properties

The Customer Service requires only minimal bootstrap configuration.

```
spring.application.name=customer-service
```

```
spring.config.import=optional:configserver:http://localhost:8888
```

The remaining properties are loaded from the Config Server.

customer-service.properties

Inside

C:\config-repo

Create

```
customer-service.properties
```

Example

```
server.port=8082
```

```
spring.datasource.url=jdbc:mysql://localhost:3306/customerdb
```

```
spring.datasource.username=root
```

```
spring.datasource.password=root
```

```
spring.jpa.hibernate.ddl-auto=update
```

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
```

When Customer Service starts, Config Server provides these properties.

Registering with Eureka

Add Eureka Client dependency.

Configure

eureka.client.service-url.defaultZone=http://localhost:8761/eureka

During startup,

Customer Service automatically registers with Eureka.

Registration Information

Eureka stores

- Application Name
- Host
- Port
- Instance ID
- Status

Example

Application

CUSTOMER-SERVICE

Status

UP

Verifying Registration

Start

- Config Server
- Eureka Server
- Product Service
- Customer Service

Open

http://localhost:8761

Expected result

Applications

PRODUCT-SERVICE

CUSTOMER-SERVICE

Status

UP

Testing CRUD APIs

Use Postman or Swagger UI.

Create Customer

POST /customers

Retrieve All Customers

GET /customers

Retrieve Customer

GET /customers/{id}

Update Customer

PUT /customers/{id}

Delete Customer

DELETE /customers/{id}

Verify that each operation updates the MySQL database correctly.

Service-to-Service Communication using RestTemplate

Spring Boot 4.1.0 | Eureka | RestTemplate | Synchronous Communication

4.1 Why Do Microservices Communicate?

In a Microservices architecture, every service is responsible for a single business capability.

Since each service owns only its own data, services must communicate to complete a business transaction.

Consider an E-Commerce application.

When a customer places an order, multiple services work together.

- Customer Service verifies customer details.
- Product Service provides product information.
- Inventory Service checks stock availability.
- Payment Service processes payment.
- Notification Service sends order confirmation.

No single service contains all the required information.

Real-World Example

Suppose a customer wants to purchase a laptop.

The application needs:

- Customer Information
- Product Details
- Stock Availability
- Payment Status

Each piece of information belongs to a different microservice.

Customer

|

Order Service

|—————▶ Customer Service

|—————▶ Product Service

└───▶ Inventory Service

└───▶ Payment Service

Each service communicates through REST APIs.

Service Independence

Each microservice has:

- Its own database
- Its own business logic
- Independent deployment
- Independent scaling

Therefore,

Customer Service should never access the Product Database directly.

Instead,

Customer Service calls Product Service through an HTTP REST API.

Incorrect Design

✗ Never do this:

Customer Service

↓

Product Database

This creates tight coupling between services.

Correct Design

Always communicate through REST APIs.

Customer Service

↓

Product Service API

↓

Product Database

This keeps every service independent.

4.2 Synchronous Communication

RestTemplate provides **Synchronous Communication**.

This means:

1. Customer Service sends a request.
2. Product Service processes the request.
3. Product Service returns a response.
4. Customer Service waits until the response is received.

Customer Service

|

HTTP Request

▼

Product Service

|

HTTP Response

▼

Customer Service

Characteristics of Synchronous Communication

- Immediate response
 - Simple implementation
 - Easy to understand
 - Uses HTTP protocol
 - Suitable for CRUD operations
-

Common Use Cases

RestTemplate is commonly used for:

- Retrieve Product Details
 - Retrieve Customer Details
 - Validate Customer
 - Verify Inventory
 - Calculate Shipping Cost
 - Retrieve Payment Status
-

4.3 What is RestTemplate?

RestTemplate is a synchronous HTTP client provided by Spring.

It allows one Spring Boot application to invoke REST APIs exposed by another application.

It automatically:

- Sends HTTP requests
 - Receives HTTP responses
 - Converts JSON into Java Objects
 - Handles HTTP methods
-

Supported HTTP Methods

RestTemplate supports all standard HTTP operations.

Method Purpose

GET Retrieve Data

Method Purpose

POST Create Resource

PUT Update Resource

DELETE Delete Resource

Advantages of RestTemplate

- Easy to use
 - Supports REST APIs
 - Automatic JSON conversion
 - Integrated with Spring Boot
 - Supports all HTTP methods
 - Works with Eureka Service Discovery
-

4.4 RestTemplate Architecture

Customer Service

|



RestTemplate

|



PRODUCT-SERVICE

|



Product Database

RestTemplate acts as the HTTP client.

Configuring RestTemplate

Create a configuration class.

@Configuration

```
public class AppConfig{
```

```
    @Bean
```

```
    @LoadBalanced
```

```
    public RestTemplate restTemplate() {
```

```
        return new RestTemplate();
```

```
    }
```

```
}
```

Why Create a Bean?

Instead of creating

```
new RestTemplate();
```

everywhere,

Spring manages one shared RestTemplate object.

Benefits:

- Reusable
 - Easier Testing
 - Better Performance
 - Cleaner Code
-

Why @LoadBalanced?

Without

@LoadBalanced

the application can only call

http://localhost:8081/api/products

This creates dependency on a fixed port.

With @LoadBalanced

Now the application can call

http://PRODUCT-SERVICE/api/products

Spring Cloud automatically asks Eureka:

"Where is PRODUCT-SERVICE?"

Eureka returns the current service instance.

Complete Flow

Customer Service

|

RestTemplate

|

Eureka Server

|

Returns Product Service Address

|

Product Service

|

JSON Response

|

Customer Service

No hardcoded host names are required.

Injecting RestTemplate

Inject RestTemplate into the Service layer using Constructor Injection.

@Service

```
public class CustomerService {  
  
    private final RestTemplate restTemplate;  
  
    public CustomerService(RestTemplate restTemplate) {  
        this.restTemplate = restTemplate;  
    }  
  
}
```

Constructor Injection is recommended because it improves:

- Readability
- Testability
- Immutability

Calling Product Service

Suppose Customer Service wants Product ID 101.

Request URL

http://PRODUCT-SERVICE/api/products/101

Using RestTemplate


```
Product product =  
restTemplate.getForObject(  
url,  
Product.class  
);
```

Spring automatically converts JSON into a Java object.

Request Flow

Client

↓

CustomerController

↓

CustomerService

↓

RestTemplate

↓

Eureka

↓

Product Service

↓

Product Database

↓

JSON Response

↓

Customer Service

↓

Client

Common RestTemplate Methods

GET

Retrieve data

`getForObject()`

POST

Create resource

`postForObject()`

PUT

Update resource

`put()`

DELETE

Delete resource

`delete()`

Using ResponseEntity

Sometimes we need additional information.

Instead of

`getForObject()`

use

`getForEntity()`

ResponseEntity provides:

- Response Body
- HTTP Status Code
- Response Headers

This is useful for advanced REST API development.

Error Handling

Communication between services may fail because:

- Product Service is unavailable
- Network Failure
- Timeout
- Invalid Product ID

Always handle exceptions gracefully.

Example

```
try{
```

```
// Call Product Service
```

```
}
```

```
catch(Exception ex){
```

```
throw new RuntimeException(
```

```
"Unable to retrieve Product");
```

}

API Documentation using OpenAPI and Swagger UI

5.1 Why API Documentation?

A REST API is useful only if other developers know:

- What endpoints are available?
- What request should be sent?
- What response will be returned?
- Which HTTP method should be used?
- Which parameters are required?
- What errors may occur?

Without documentation, developers spend significant time reading source code or relying on external documentation.

Example Without Documentation

Suppose Customer Service exposes an endpoint.

GET /customers/101

Questions immediately arise:

- What does 101 represent?
- What fields are returned?
- What happens if the customer does not exist?
- Is authentication required?
- What response codes are returned?

Without documentation, developers have to guess or inspect the implementation.

Problems Without API Documentation

Lack of documentation can lead to:

- Increased development time
- Communication gaps between teams
- Incorrect API usage

- Difficult testing
 - Poor developer experience
 - Higher maintenance effort
-

5.2 What is OpenAPI?

OpenAPI is an industry-standard specification for describing REST APIs.

It provides a machine-readable description of:

- Available endpoints
- HTTP methods
- Request parameters
- Request body
- Response body
- Response codes
- API metadata

Spring applications can generate this documentation automatically.

What Does OpenAPI Generate?

OpenAPI automatically creates documentation such as:

GET /customers

GET /customers/{id}

POST /customers

PUT /customers/{id}

DELETE /customers/{id}

Developers can immediately understand the available operations without reading the source code.

5.3 What is Swagger UI?

Swagger UI is a web application that displays OpenAPI documentation in a user-friendly interface.

Instead of reading static documents, developers can:

- View available APIs
- Understand request parameters
- Execute APIs
- View responses
- Test endpoints interactively

OpenAPI Architecture

Developer

|

Starts Application

|

Spring Boot

|

OpenAPI

|

Generates API Specification

|

Swagger UI

|

Interactive Documentation

Documentation is generated automatically whenever the application starts.

Benefits of Swagger UI

- Interactive documentation
- Automatic updates
- No manual documentation
- Easy API testing
- Better collaboration
- Faster onboarding of developers
- Improved API understanding

5.4 Adding OpenAPI Dependency

Add the Springdoc OpenAPI dependency to the project.

For **Spring Boot 4.1.0**, use the version compatible with the current Spring Boot release.

Example Maven dependency:

```
<dependency>
<groupId>org.springdoc</groupId>
<artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
<version>2.8.9</version>
</dependency>
```

Starting the Application

Once the dependency is added,
start Customer Service.

Springdoc automatically scans:

- @RestController
- Request Mapping annotations

- Response types
- Validation annotations

No additional coding is required for basic documentation.

Swagger UI URL

Open your browser and navigate to

<http://localhost:8082/swagger-ui.html>

or

<http://localhost:8082/swagger-ui/index.html>

Depending on the Springdoc version being used.

Example Swagger Screen

Customer Controller

GET /customers

GET /customers/{id}

POST /customers

PUT /customers/{id}

DELETE /customers/{id}

Each endpoint can be expanded to view details.

API Information

Swagger UI displays:

- API Name
- Description
- HTTP Method
- URL

- Parameters
- Request Body
- Response Body
- Response Codes

Developers no longer need separate documentation.

Try It Out Feature

Swagger UI provides a **Try it out** button.

Example:

POST /customers

Enter JSON

```
{  
  "firstName": "Rahul",  
  "lastName": "Sharma",  
  "email": "rahul@gmail.com",  
  "phone": "9876543210",  
  "city": "Kanpur"  
}
```

Click

Execute

Swagger sends the HTTP request and immediately displays the response.

Understanding HTTP Response Codes

Typical responses include:

Status Code Meaning

200	Success
201	Resource Created
400	Invalid Request
404	Resource Not Found
500	Internal Server Error

Swagger clearly displays these responses during testing.

Testing Customer APIs

Using Swagger UI, verify the following operations:

Create Customer

POST /customers

Expected Result

- Customer created successfully
 - Status 201 (or 200, depending on implementation)
-

Retrieve All Customers

GET /customers

Expected Result

- List of customers returned
-

Retrieve Customer by ID

GET /customers/{id}

Expected Result

- Customer details returned
 - 404 if customer does not exist
-

Update Customer

PUT /customers/{id}

Expected Result

- Customer information updated
-

Delete Customer

DELETE /customers/{id}

Expected Result

- Customer removed successfully
-

Advantages of OpenAPI

- Automatic documentation
- No manual maintenance
- Interactive testing
- Better collaboration
- Easier debugging
- Faster development
- Improved API consistency